



XÂY DỰNG TRÒ CHƠI RẮN SẴN MỖI



SHAKE GAME

Sinh viên thực hiện:
Trần Hoàng Long

Giáo viên hướng dẫn:
ThS. Khâu Văn Nhựt

MÔ TẢ ĐỒ ÁN

Đồ tài xây dựng trò chơi Rắn Sẵn Mồi trên nền tảng Unity theo hướng đối tượng, kết hợp các thuật toán tìm đường để tạo ra chế độ chơi AI có thể tự điều khiển rắn ăn mồi và hạn chế va chạm. Mục tiêu là vận dụng kiến thức OOP, cấu trúc dữ liệu, giải thuật vào một sản phẩm game 2D hoàn chỉnh, có giao diện thân thiện, dễ mở rộng và có khả năng phát triển thêm các chế độ chơi mới. Game đạt hiệu năng 60 FPS ổn định với RAM usage chỉ 150-200MB.

CÔNG CỤ VÀ CÔNG NGHỆ



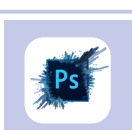
Unity 2025
LTS



C# 9.0
Programming



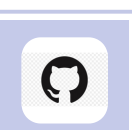
ShaderLab & HLSL



Adobe
Photoshop



Visual Paradigm



GitHub Version
Control

NỘI DUNG VÀ CHỨC NĂNG

- Hệ thống Menu đa tầng: Thiết kế giao diện menu chính với các lựa chọn chế độ chơi (Single Player / Player vs AI), màn hình cài đặt tùy chỉnh, bảng xếp hạng lưu trữ top điểm cao, và hệ thống Pause/Resume trong game.
- Core Gameplay nâng cao: Xây dựng bộ điều khiển rắn theo OOP với Input Buffering System tránh hiện tượng "nuốt phím" khi thao tác nhanh, xử lý logic ăn mồi tăng trưởng, hệ thống Combo tính điểm thưởng khi ăn liên tiếp.
- Grid System chính xác: Cài đặt lưới tọa độ 40x30 cells với HashSet cho collision detection O(1), thay thế Physics2D để loại bỏ sai số floating point, đảm bảo di chuyển chính xác tuyệt đối trên grid.
- AI thông minh hai chiến thuật: Phát triển AI sử dụng BFS tìm đường ngắn nhất (Hunting Mode) khi có path an toàn, và Flood Fill đánh giá vùng trống (Survival Mode) khi bị bao vây, tự động đồng bộ tốc độ với người chơi.
- Hệ thống sinh vật phẩm thông minh: Thuật toán spawn thức ăn với 3 độ hiếm (Common 70%, Rare 25%, Epic 5%), áp dụng Manhattan Distance đảm bảo không sinh quá gần rắn, tạo cân bằng gameplay.
- Tối ưu hiệu năng: Áp dụng Object Pooling cho Audio System giảm GC Allocation, Singleton Pattern cho các Manager, sử dụng ScriptableObject tách data khỏi logic, đạt 60 FPS ổn định.
- VFX & Animation toán học: Hiệu ứng Neon Style với Glow/Outline, Procedural Animation dùng hàm Sin/Cos cho floating và pulse effect, Camera thích ứng tự động điều chỉnh theo tỷ lệ màn hình (4:3, 16:9, 21:9).
- Đa ngôn ngữ & Lưu trữ: Hệ thống Localization hỗ trợ Tiếng Việt/Tiếng Anh chuyển đổi tức thì, Data Persistence với JSON lưu trữ High Score và Settings, hệ thống 10 bộ màu Snake Skin tùy chỉnh.

THUẬT TOÁN SỬ DỤNG

BFS (Breadth-First Search) – Tìm đường đi ngắn nhất đến thức ăn với độ phức tạp $O(V \cdot E) \approx O(1)$ trên grid 40x30.

Flood Fill – Ước lượng vùng trống còn lại, hạn chế trường hợp rắn tự khóa khi bị bao vây.

Manhattan Distance – Heuristic function cho pathfinding chính xác trên grid-based movement.

Input Buffering – Xử lý hàng đợi phím bấm, giúp điều khiển mượt mà mà không bị skip input.

THỐNG KÊ VÀ MÃ NGUỒN

Cơ cấu mã nguồn (GitHub Repository): C# ~ 50.4% | ShaderLab ~ 41.1% | HLSL ~ 8.5%

Repository: CSN-TranHoangLong-110123025-SnakeGame

Branch: new (latest development)

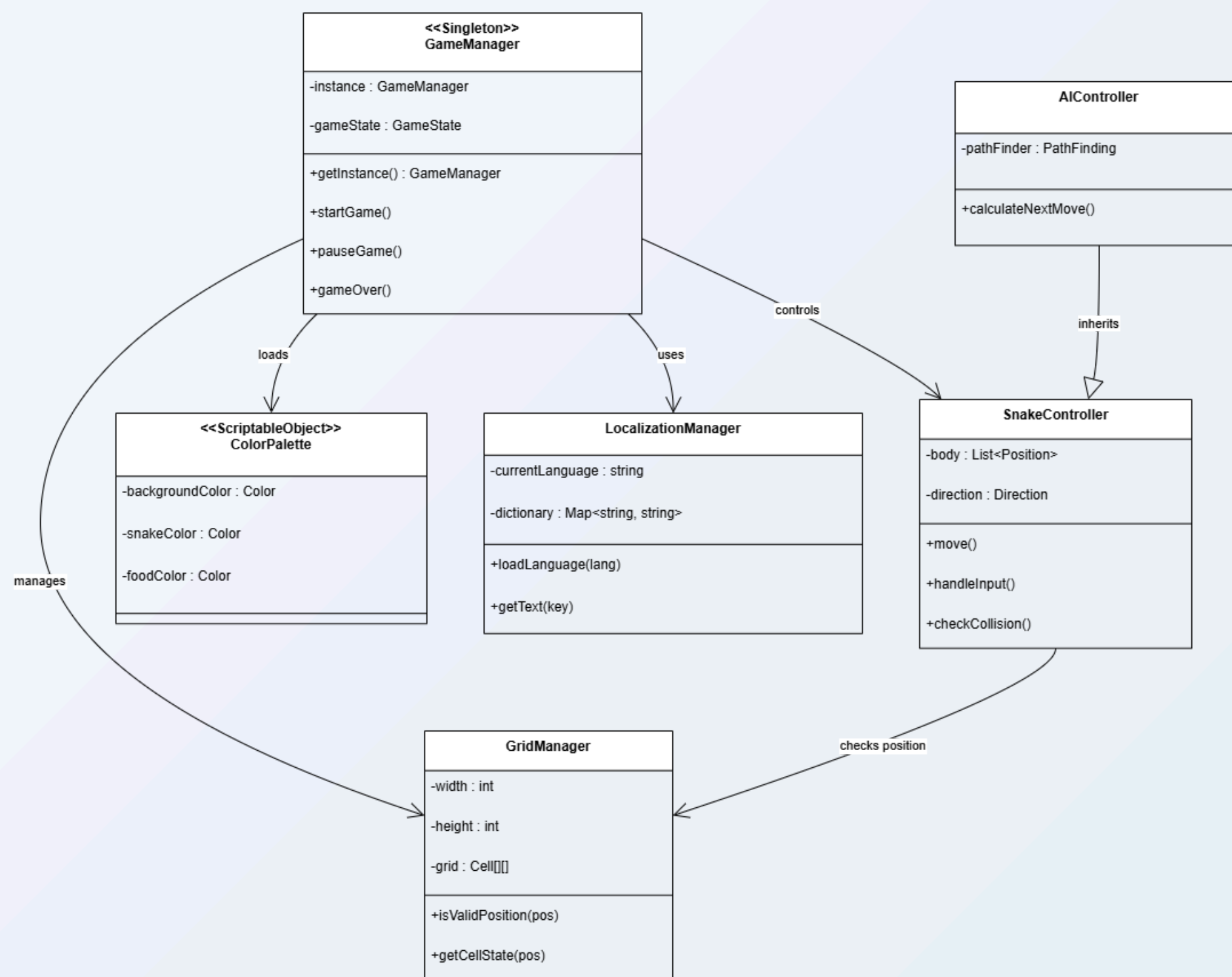
Hiệu năng: 60 FPS | RAM: 150-200MB | Load time: <2s



Quét mã để tải game và xem mã nguồn:

github.com/trhlow/CSN-TranHoangLong-110123025-SnakeGame

KIẾN TRÚC HỆ THỐNG



ĐẶC ĐIỂM NỔI BẬT

Grid-based Collision O(1) – Sử dụng HashSet thay vì List để kiểm tra va chạm, giảm độ phức tạp từ O(N) xuống O(1).

Input Buffering System – Xử lý hàng đợi phím bấm, tránh hiện tượng "nuốt phím" khi thao tác nhanh.

Object Pooling cho Audio – Tái sử dụng AudioSource, giảm GC Allocation và tối ưu hiệu năng.

Camera thích ứng – Tự động điều chỉnh vùng nhìn theo tỷ lệ màn hình (4:3, 16:9, 21:9).

Procedural Animation – Hiệu ứng chuyển động được tính toán theo thời gian thực bằng hàm Sin/Cos.

Data Persistence với JSON – Lưu trữ bảng xếp hạng và cài đặt người dùng dạng JSON.

KẾT QUẢ ĐẠT ĐƯỢC

